MI MIPITX API

Version 2.01

REVISION HISTORY

Revision No.	Description	Date
2.00	• Initial release	12/18/2019
2.01	• Added new API: MI_ MipiTx_InitDev and MI_ MipiTx_DeInitDev	05/08/2020

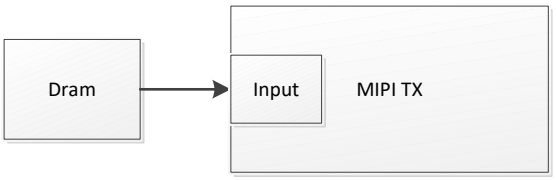
TABLE OF CONTENTS

1. SCOPE

1.1. Module Description

The MIPITX module is used to send the DRAM data through MIPI interface protocol.

1.2. Flow Chart



1.3. Keyword

N/A

2. API LIST

This module provides the following APIs:

Name of API	Function
MI_MipiTx_CreateChannel	Create a MipiTx channel
MI_MipiTx_DestroyChannel	Destroy a MipiTx channel
MI_MipiTx_GetChannelAttr	Get a MipiTx channel attribute
MI_MipiTx_StartChannel	Start a MipiTx channel
MI_MipiTx_StopChannel	Stop a MipiTx channel
MI_MipiTx_SetTimingConfig	Set the MipiTx Channel Timing parameter
MI_MipiTx_GetTimingConfig	Get the MipiTx Channel Timing parameter
MI_MipiTx_InitDev	Initialize MipiTx device
MI_MipiTx_DeInitDev	De-initialize MipiTx device

2.1. MI_MipiTx_CreateChannel

- Description

  Create a MipiTx channel.

- Syntax

  MI_S32 MI_MipiTx_CreateChannel(MI_U32 u32ChannelId, MI_MipiTx_ChannelAttr_t *pstMipiTxChAttr);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input
pstMipiTxChAttr	MipiTx channel attribute pointer.	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note
  - Multi-channel creation is not supported

- Example

The Mipi TX function setting is as follows：

```
MI_U32 u32ChnId = 0;
MI_MipiTx_ChannelAttr_t stMipiChnAttr;
memset(&stMipiChnAttr, 0x0, sizeof(MI_MipiTx_ChannelAttr_t));

stMipiChnAttr.eLaneNum = E_MI_MIPITX_LANE_NUM_4;
stMipiChnAttr.ePixFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
stMipiChnAttr.u32Dclk = 445500;
stMipiChnAttr.u8DCLKDelay = 0;
stMipiChnAttr.u32Width = 1920;
stMipiChnAttr.u32Height = 1080;
stMipiChnAttr.peChSwapType = NULL;

MI_MipiTx_CreateChannel(u32ChnId, &stMipiChnAttr);

MI_MipiTx_TimingConfig_t stTimingCfg;
memset(&stTimingCfg, 0x0, sizeof(MI_MipiTx_TimingConfig_t));

stTimingCfg.u8Lpx  = 0x07;
stTimingCfg.u8ClkHsPrpr = 0x08;
```

```
stTimingCfg.u8ClkZero = 0x20;
stTimingCfg.u8ClkHsPre = 0x01;
stTimingCfg.u8ClkHsPost = 0x10;
stTimingCfg.u8ClkTrail = 0x0a;
stTimingCfg.u8HsPrpr = 0x01;
stTimingCfg.u8HsZero = 0x0F;
stTimingCfg.u8HsTrail = 0X0A;

MI_MipiTx_SetTimingConfig(u32ChnId, &stTimingCfg);

MI_MipiTx_StartChannel(u32ChnId);

/****exit****/
MI_MipiTx_StopChannel(u32ChnId);
MI_MipiTx_DestroyChannel(u32ChnId);
```

Note: For DCLK parameter setting, please refer to the note in MI_MipiTx_ChannelAttr_t.
For timing parameter setting, please refer to the note in MI_MipiTx_TimingConfig_t.

- Related APIs

  MI_MipiTx_DestroyChannel

2.2. MI_MipiTx_DestroyChannel

- Description

  Destroy a MipiTx channel.

- Syntax

  MI_S32 MI_MipiTx_DestroyChannel(MI_U32 u32ChannelId);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel

- Related APIs

  MI_MipiTx_CreateChannel

2.3. MI_MipiTx_GetChannelAttr

- Description

  Get a MipiTx channel attribute

- Syntax

  MI_S32 MI_MipiTx_GetChannelAttr(MI_U32 u32ChannelId, MI_MipiTx_ChannelAttr_t *pstMipiTxChAttr);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input
pstMipiTxChAttr	MipiTx channel attribute pointer.	Output

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel

- Related APIs

  MI_MipiTx_CreateChannel

2.4. MI_MipiTx_StartChannel

- Description

  Start a MipiTx channel.

- Syntax

  MI_S32 MI_MipiTx_StartChannel(MI_U32 u32ChannelId);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel

- Related APIs

  MI_MipiTx_StopChannel

2.5. MI_MipiTx_StopChannel

- Description

  Stop a MipiTx channel.

- Syntax

  MI_S32 MI_MipiTx_StopChannel(MI_U32 u32ChannelId);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel.

- Related APIs

  MI_MipiTx_StartChannel

2.6. MI_MipiTx_SetTimingConfig

- Description

  Set the MipiTx channel timing parameter

- Syntax

  MI_S32 MI_MipiTx_SetTimingConfig(MI_U32 u32ChannelId, MI_MipiTx_TimingConfig_t *pstMipiTimingCfg);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input
pstMipiTimingCfg	MipiTx timing parameter	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel.

- Related APIs

  MI_MipiTx_GetTimingConfig

2.7. MI_MipiTx_GetTimingConfig

- Description

  Get the MipiTx channel timing parameter

- Syntax

  MI_S32 MI_MipiTx_GetTimingConfig(MI_U32 u32ChannelId, MI_MipiTx_TimingConfig_t *pstMipiTimingCfg);

- Parameters

Parameter Name	Description	Input/Output
u32ChannelId	MipiTx channel number.	Input
pstMipiTimingCfg	MipiTx timing parameter.	Output

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note

  None

- Example

  Refer to MI_MipiTx_CreateChannel.

- Related APIs

  MI_MipiTx_SetTimingConfig.

2.8. MI_MipiTx_InitDev

- Description

  Initialize MipiTx device

- Syntax

  MI_S32 MI_MipiTx_InitDev(MI_MipiTx_InitParam_t *pstInitParam);

- Parameters

Parameter Name	Description	Input/Output
pstInitParam	Initialization Parameter	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note
  - This interface should be used in pair with MI_MipiTx_DeInitDev; otherwise, a failed message will be returned.

- Example

  None

- Related APIs

  None

2.9. MI_MipiTx_DeInitDev

- Description

  De-Initialize MipiTx device

- Syntax

  MI_S32 MI_MipiTx_DeInitDev(void);

- Parameters

  NULL

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependence
  - Header file: mi_mipitx.h, mi_mipitx_datatype.h
  - Lib: libmi_mipitx.a / libmi_mipitx.so

- Note
  - This interface should be called after device has been initialized; otherwise, a failed message will be returned.
  - This interface should be used in pair with MI_MipiTx_InitDev; otherwise, a failed message will be returned.

- Example

  None

- Related APIs

  None

3. MIPITX DATA TYPE

MipiTx module related data types are listed below:

Data Type	Definition
MI_MipiTx_TimingConfig_t	Define MipiTx timing type
MI_MipiTx_LaneNum_e	Define MipiTx lane num data
MI_MipiTx_ChannelSwapType_e	Define Lane ID
MI_MipiTx_ChannelAttr_t	Define MipiTx channel attribute
MI_MipiTx_InitParam_t	Define the initialized parameter of MipiTx device

3.1. MI_MipiTx_TimingConfig_t

- Description

  Define MipiTx timing type

- Definition

```
typedef struct MI_MipiTx_TimingConfig_s
{
    MI_U8 u8Lpx;
    MI_U8 u8ClkHsPrpr;
    MI_U8 u8ClkZero;
    MI_U8 u8ClkHsPre;
    MI_U8 u8ClkHsPost;
    MI_U8 u8ClkTrail;
    MI_U8 u8HsPrpr;
    MI_U8 u8HsZero;
    MI_U8 u8HsTrail;
} MI_MipiTx_TimingConfig_t;
```

- Member

Member Name	Description
u8ClkHsPost	Clock lane maintains the high-speed clock state time after all data lanes enter the low-power mode. See $T_{CLK-POST}$ in Figure 3.1.2
u8ClkTrail	Clock lane maintains HS-0 time before entering low power mode. See $T_{CLK-TRAIL}$ in Figure 3.1.2
u8Lpx	Clock lane maintains LP-01 time in low-power mode. See T_{LPX} in Figure 3.1.2.
u8ClkHsPrpr	Clock lane maintains LP-00 time in low-power mode. See $T_{CLK-PREPARE}$ in Figure 3.1.2.
u8ClkZero	Clock lane maintains HS-0 time before entering high-speed mode. See $T_{CLK-ZERO}$ in Figure 3.1.2.
u8ClkHsPre	Clock lane maintains the high-speed clock state time before data transmission. See $T_{CLK-PRE}$ in Figure 3.1.2.
u8HsPrpr	Data lane maintains LP-00 time before data transmission. See $T_{HS-PREPARE}$ in Figure 3.1.1.
u8HsZero	Data lane maintains HS-0 time before data transmission. See $T_{HS-ZERO}$ in Figure 3.1.1.
u8HsTrail	Data lane maintains HS-0 or HS-1 time after data transmission. See $T_{HS-TRAIL}$ in Figure 3.1.1.

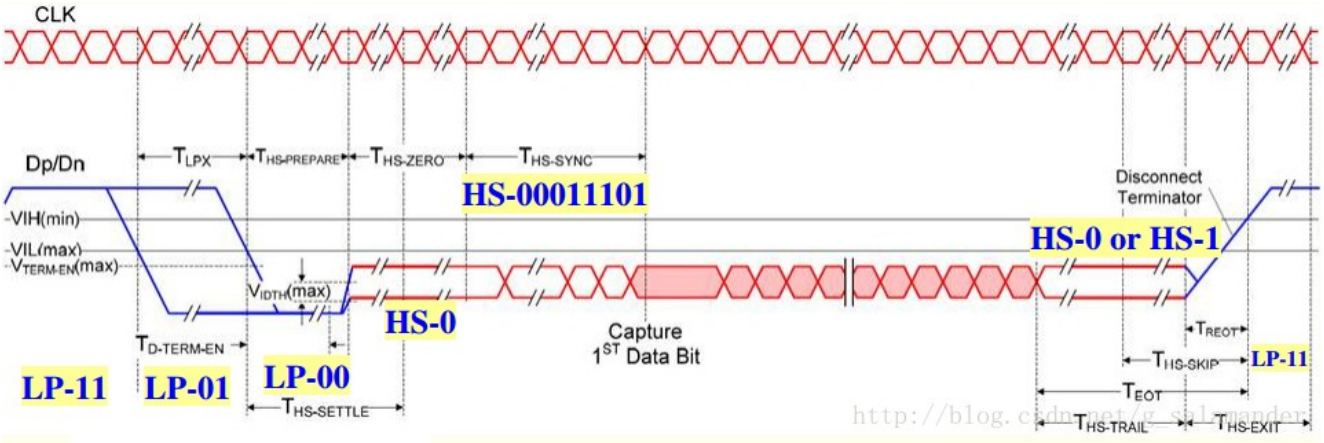


Figure 3.1.1: Time sequence from low-power mode (LP) to high-speed mode (HS)

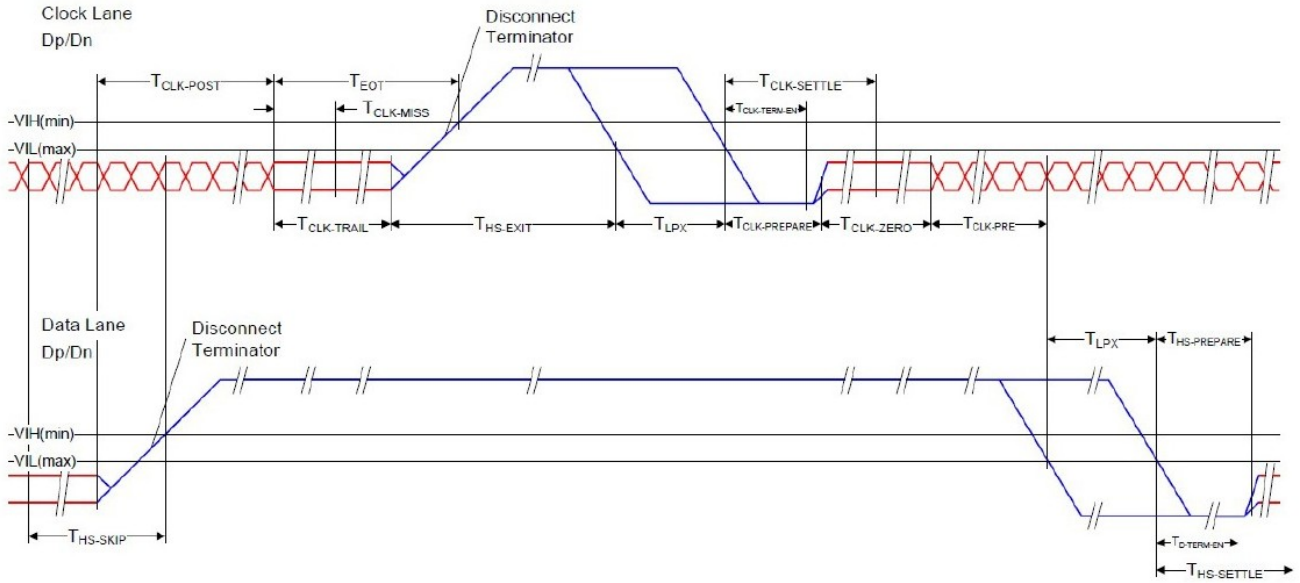


Figure 3.1.2: Time sequence from high-speed mode to low-power mode

- Note

Timing (T)	Default	Default @1Gbps (T_B =8ns)	MIPI D-PHY Spec. (v1.2)		
			Min	Typ	Max
T_{LPX}	$8T_B$	64ns	50ns		
$T_{CLK-PREPARE}$	$9T_B$	72ns	38ns		95ns
$T_{CLK-ZERO}$ + $T_{CLK-PREPARE}$	$33T_B$+$9T_B$	336ns	300ns		
$T_{CLK-PRE}$	$2T_B$	16ns	8ns		
$T_{CLK-POST}$	$17T_B$	136ns	60ns + 52UI		
$T_{CLK-TRAIL}$	$11T_B$	88ns	60ns		
$T_{HS-PREPARE}$	$10T_B$	80ns	40ns+4UI		85ns+6UI
$T_{HS-ZERO}$ + $T_{HS-PREPARE}$	$16T_B$+$10T_B$	208ns	145ns+10UI		
$T_{HS-TRAIL}$	$11T_B$	88ns	Max(8UI, 60ns+4UI)		

  - Timing config referred in the table above, computing formula is Time = (R + 1) * TB, Time match Default column, R is demand. For example, calculate u8Lpx, Known Time=8, result R=7, so u8Lpx=7.

- Related Type

  MI_MipiTx_SetTimingConfig
  MI_MipiTx_GetTimingConfig

3.2. MI_MipiTx_LaneNum_e

- Description

  Define MipiTx Lane num data

- Definition

```
typedef enum
{
    E_MI_MIPITX_LANE_NUM_NONE = 0,
    E_MI_MIPITX_LANE_NUM_1    = 1,
    E_MI_MIPITX_LANE_NUM_2    = 2,
    E_MI_MIPITX_LANE_NUM_3    = 3,
    E_MI_MIPITX_LANE_NUM_4    = 4,
} MI_MipiTx_LaneNum_e;
```

- Note

  - Mipi Tx and Rx must have the same number of lane num.

- Related Type

  MI_MipiTx_ChannelAttr_t.

3.3. MI_MipiTx_ChannelSwapType_e

- Description

  Define Lane ID

- Definition

```
typedef enum
{
    E_MI_MIPITX_CH_SWAP_0,
    E_MI_MIPITX_CH_SWAP_1,
    E_MI_MIPITX_CH_SWAP_2,
    E_MI_MIPITX_CH_SWAP_3,
    E_MI_MIPITX_CH_SWAP_4,
} MI_MipiTx_ChannelSwapType_e;
```

- Note
  - By default, lane0 CH0, lane1 ch1, lane2 CH2, lane3 CH3 and CLK CH4 correspond.

- Related Type

  MI_MipiTx_ChannelAttr_t.

3.4. MI_MipiTx_ChannelAttr_t

- Description

  Define MipiTx Channel attribute

- Definition

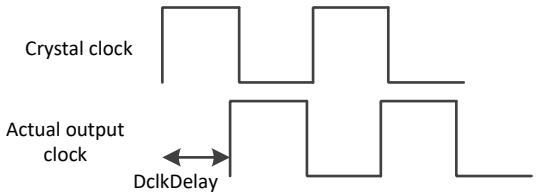
```
typedef struct
{
    MI_U32                  u32Width;
    MI_U32                  u32Height;
    MI_SYS_PixelFormat_e        ePixFormat;
    MI_MipiTx_LaneNum_e         eLaneNum;

    MI_U8                   u8DCLKDelay; ///<  DCLK Delay
    MI_U32                  u32Dclk;     ///<  DCLK ( Htt * Vtt * Fps)

    MI_MipiTx_ChannelSwapType_e    *peChSwapType;  ///< all lane order swap
}MI_MipiTx_ChannelAttr_t;
```

- Member

Member Name	Description
u32Width	Transmit data level size
u32Height	Transmit data vertical size
ePixFormat	Transmit data pixel format
enLaneNum	Number of transmit data channels
u8DCLKDelay	Transmit data clock skew
u32Dclk	Transmit data clock
peChSwapType	Lane ID order

➢ Note

- ePixFormat currently only supports YUV422 and RAW8 format.
- u32Dclk = width*1.2*height*1.2*Fps*bit/LaneNum unit HZ. There are four gears in all: 100M~200M, 200M~400M, 400M~800M, and 800M~1500M.
- u8DCLKDelay default is 0, no setup is required, and clock skew is adjusted after prevention.



- When peChSwapType == NULL, chn 0~4 and lane 0~4 correspond in turn by default. Otherwise, peChSwapType needs to specify the lane ID corresponding to each CHN.

➢ Related Type

MI_MipiTx_CreateChannel

3.5. MI_MipiTx_InitParam_t

➢ Description

MipiTx device initialization parameter.

➢ Definition

```
typedef struct MI_MipiTx_InitParam_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_MipiTx_InitParam_t;
```

➢ Member

Member Name	Description
u32DevId	Device ID
u8Data	Pointer of data buffer

➢ Note

NULL

➢ Related data types and interfaces

MI_MipiTx_InitDev

4. ERROR CODE

MipiTx API error code is defined in the table below:

Table 1: MipiTx API Error Code

Error Code	Macro Definition	Description
0xA01E1016	MI_ERR_MIPITX_CHN_HAVECREATE	Channel created
0xA01E1002	MI_ERR_MIPITX_CHNID_INVALID	Channel Id illegal
0xA01E1014	MI_ERR_MIPITX_CHN_NOTSTOP	Channel not stopped
0xA01E1003	MI_ERR_MIPITX_ILLEGAL_PARAM	Illegal parameter
0xA01E2004	MI_ERR_MIPITX_RUNFAIL	Function execution failed
0xA01E2006	MI_ERR_MIPITX_NULL_PTR	Input parameter null pointer error